

PERPETUAL MOTION SQUAD

PocketExpense+

Production-Grade Offline-First Financial Tracking System

Automated SMS transaction detection · Offline-first sync · Smart budgeting

Purvansh Sahu · Yash Arya · Aman Anand

BSc (Hons) Computer Science · BITS Pilani · Guide: Mr. Murali Raghu · August 2026

Agenda

- Problem Statement
- Solution Overview
- System Architecture
- Technology Stack
- Database Design
- The 12 Features
- SMS Detection Pipeline & Accuracy
- Security & Privacy
- Testing Strategy — 198 tests
- Performance & Reliability
- Challenges & Future Work
- Conclusion & Demo

Problem Statement

Expense-tracking applications fail for two systemic reasons:

Manual-entry fatigue

- Logging every small transaction is unsustainable — most users abandon within weeks.
- High friction → incomplete records → useless analytics.

Offline fragility

- Most apps stop working the moment connectivity drops.
- Connected aggregators (Mint, Plaid-based) require surrendering banking credentials and constant connectivity.

Mint's 2023 shutdown destroyed years of users' financial history — the aggregator model is fragile and trust-heavy.

Our goal: an expense app that *logs itself* and *works without internet*.

Solution Overview

99%

SMS detection accuracy

0%

false positives

198

automated tests passing

12/12

features complete

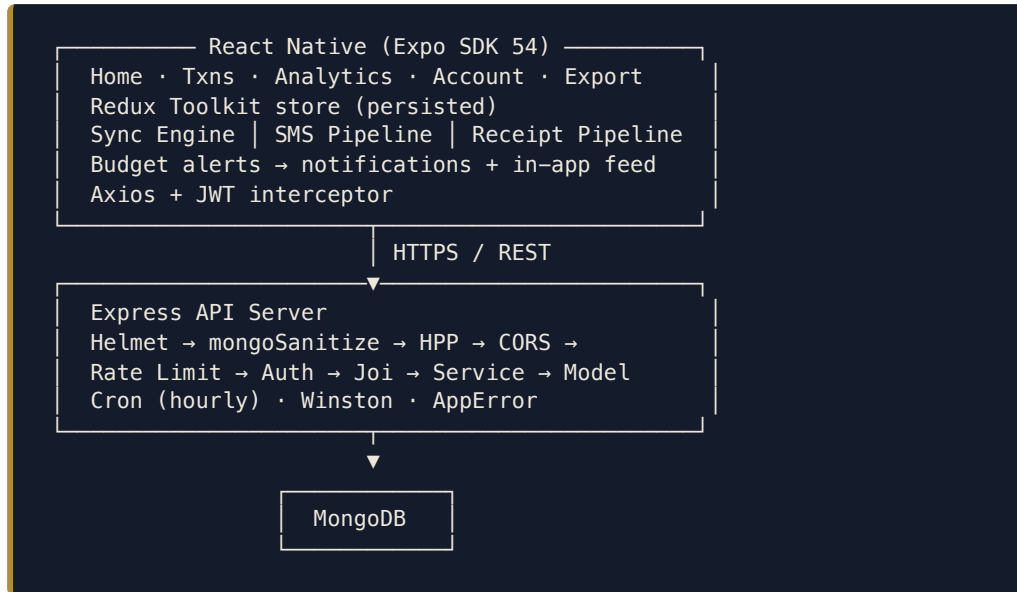
Automation

- Android SMS detection parses bank messages entirely on-device.
- Recurring transactions processed hourly, idempotently.
- Receipt OCR prefills the expense form from a photograph.

Offline-first

- Every operation works locally; sync is automatic and resilient.
- Proactive budget alerts at 80% / 100%.
- Privacy: no SMS content ever leaves the device.

System Architecture



Key design points

- **Thick client:** local persistence, sync, SMS/OCR parsing on-device.
- **Layered backend:** controllers thin, services own all logic.
- **Uniform API envelope:** { success, message, data }.
- **Offline-first:** tombstones, exponential backoff, paged sync.

Technology Stack

Layer	Technology	Version
Mobile	React Native + Expo	0.81.5 / SDK 54
Routing / State	Expo Router / Redux Toolkit	6.0.23 / 2.11.2
Persistence	AsyncStorage + persistMiddleware	2.2.0
Backend	Node.js + Express	20+ / 4.18.2
Database	MongoDB + Mongoose	8.0.3
Auth	JWT + bcrypt (10 rounds)	9.0.2 / 2.4.3
Validation / Security	Joi / Helmet, mongo-sanitize, hpp, rate-limit	18.0.2
Testing / Load	Jest · Supertest · mongodb-memory-server · Artillery	30.2.0

Database Design

User ← Expense ← Budget

User	Expense	Budget
<code>_id</code> PK	<code>_id</code> PK	<code>_id</code> PK
<code>name</code> <code>email</code>	<code>user</code> FK	<code>user</code> FK
<code>password</code>	<code>amount</code>	<code>category</code>
<code>budgetLimit</code>	<code>type</code>	<code>amount</code>
<code>currency</code>	<code>category (20)</code>	<code>month</code> · <code>year</code>
<code>avatar</code>	<code>paymentMethod</code>	<code>totalSpent</code>
	<code>date</code>	virtuals:
	<code>isRecurring</code>	<code>percentageUsed</code>
	<code>frequency</code>	<code>remainingAmount</code>
	<code>localId</code>	
	<code>syncStatus</code>	

Indexes

- **Expense (6):** `user+date` · `user+category` · `user+type+date` · combined filters · `isRecurring+nextRunDate (cron)` · `user+localId (sync)`
- **Budget (2):** `user+category+month+year` **UNIQUE** · `user+month+year`

Offline sync fields

- `localId` — stable client-side UUID
- `syncStatus` — synced / pending / conflict

The 12 Features

#	Feature	#	Feature
1	JWT authentication	7	Receipt scanning + OCR prefill
2	Expense CRUD + offline sync + bulk delete	8	Android SMS detection + auto-add with Undo
3	Category budgets + threshold alerts	9	Export transactions (CSV / PDF)
4	Recurring transactions (hourly cron, idempotent)	10	Notifications (local + in-app feed)
5	Advanced filtering + pagination	11	Theme system (light / dark / system)
6	Smart insights (growth, top categories, anomalies)	12	Production hardening (security, rate limits, logging)

All twelve features are **complete and tested** — covered by 198 automated tests.

SMS Detection Pipeline (Android)

```

Incoming bank SMS
| smsListener (100ms debounce)
|
1. normalizeMessage      strip zero-width chars, ₹/INR
2. detectSenderMetadata  40+ bank sender-ID patterns
3. classifyMessageType    debit / credit / ignore
4. extractFields         amount, merchant, account, ts
5. computeConfidence     weighted 0.0-1.0 scoring
|
Deduplication LRU cache (200) + 2-min guard
|
Category auto-detection 100+ keyword → category
|
Routing ≥0.75 modal · <0.50 ignore ·
        auto-add ≥0.9 with 6s Undo
  
```

Confidence scoring

- Bank sender +0.25 · amount +0.30 · merchant +0.20 · keyword +0.15 · account +0.05 · timestamp +0.05
- Penalties: ambiguous -0.20 · multi-amount -0.15 · suspicious -0.10

Guardrails

- Amount ₹1 – ₹10,00,000 · strict decimal parsing · merchant ≥ 2 chars · min SMS length 15
- Unrecognised sender below 0.65 always ignored

SMS Detection — Measured Results

99%accuracy (target $\geq 95\%$)**0%**false positives ($\leq 3\%$)**0.92**avg confidence (≥ 0.6)**100%**

OTP & promo rejection

Total samples: 200 · TP 158 · TN 40 · FP 0 · FN 2
Debit detection 100% · Credit detection 95%

- Validated against a **200-sample corpus** (120 debit · 40 credit · 20 OTP · 20 promo).
- Across 40 adversarial negatives (OTPs and adverts containing rupee amounts), **none** was recorded.
- Auto-add (≥ 0.9 confidence) logs instantly with a 6-second Undo — reversibility makes silent logging safe.

Budgets, Recurring & Insights

Budgeting

- Overall + per-category monthly budgets.
- Virtual fields: `percentageUsed`, `remainingAmount`.
- **Alerts at $\geq 80\%$ and $\geq 100\%$** , once per budget per month.
- `totalSpent` recalculated via aggregation on every change.

Recurring & Insights

- Daily / weekly / monthly; hourly cron with **idempotency guard**.
- Smart insights: 6-month growth, top-3 categories, weekday vs weekend.
- Anomaly detection via **z-score** (flags > 2 std dev).
- Home stats computed locally — consistent offline.

Offline-First Reliability

Sync engine

- **Tombstones** — offline deletes never resurrect.
- **Exponential backoff** — $2s \rightarrow \dots \rightarrow 5\text{min}$ cap.
- **Paged pull** — 100 records/page, 20-page cap.
- **Reconciliation** — $\text{localId} \rightarrow \text{serverId}$ mappings.
- Force sync via pull-to-refresh; delete retry cap = 5.

Performance targets

Metric	Target
p95 response	< 500 ms
p99 response	< 1000 ms
Concurrent users	100
Request rate	500 req/min

Artillery fails the run if a latency target is breached.

Security & Privacy

Security controls

- Helmet headers · NoSQL injection prevention · http
- Rate limiting: 100/15min API · 20/15min auth
- JWT (7-day) + bcrypt (10 rounds); password `select:false`
- Joi validation everywhere · CORS allowlist · 10MB limit
- Graceful shutdown (SIGTERM/SIGINT)

SMS privacy

- **100% on-device parsing** — raw SMS never stored or logged
- **No backend endpoint** can receive SMS content
- Off by default · revocable · minimal permissions (READ_SMS, RECEIVE_SMS)
- Dedup hashes are non-reversible fingerprints

Testing Strategy — 198 Tests

Backend — 96

Suite	Tests
Expense Service	15
Budget Service	12
Recurring Service	10
Insight Service	11
API Integration	17
Edge Cases	31

Coverage ~82% · in-memory MongoDB

Frontend — 102

Suite	Tests
SMS Parser (200-sample)	42
Receipt Parser	16
Export Format	16
Budget Thresholds	12
Bulk Delete	4
Stats	12

Plus `tsc --noEmit` and `expo lint` clean

Result: 198/198 passing · 0 failures · 0 open defects.

Challenges & Future Work

Challenges overcome

Challenge	Solution
OCR / SMS need native modules absent in Expo Go	Provider isolation + graceful fallback
Duplicate recurring entries on cron double-fire	Idempotency guard on lastProcessedDate
Offline deletes resurrecting	Tombstones propagated on sync
False positives writing wrong records	Confidence scoring + gating + Undo
Unbounded retry loops	Backoff + paged pulls + retry caps

Future enhancements

- Secure token storage (expo-secure-store)
- Multi-currency with live exchange rates
- Cloud push notifications (FCM / Expo push)
- Bull/BullMQ job queue; Redis cache; sharding
- SMS ML model (>99.5%); cloud OCR endpoint

Conclusion

- 12/12 features implemented and documented
- 198/198 automated tests · ~82% backend coverage
- SMS accuracy 99% · 0% false positives · 0.92 confidence
- Offline-first with real resilience engineering
- Production-hardened backend — approved for deployment
- Full demo available as a video walkthrough

THANK YOU — QUESTIONS?

github.com/purvanshh/PocketExpense-

Perpetual Motion Squad · BITS Pilani · August 2026